GRUPRO — Força bruta

subconjuntos, permutações, inclusão–exclusão

Jonathan Queiroz

Subconjuntos

Operações com bits (exemplos)

```
// Carrega x = 8 (0000.1000)
int x = 1 << 3;
// Desloca uma posição para a esquerda: x = 16 (0001.0000)
x = x << 1;
// Desloca duas posições para a direita: x = 4 (0000.0100)
x = x >> 2;
// Liga o bit 3: x = 12 (0000.1100)
x = x | (1<<3);
// Testa o bit 2
if (x & (1<<2)) puts("sim") else puts("nao"); // sim
// Testa o bit 0
if (x & (1<<0)) puts("sim") else puts("nao"); // nao
// Desliga o bit 2: x = 8 (0000.1000)
x = x & ~(1<<2);
// Carrega x = 15 (0000.1111)
x = (1 << 4) - 1;
```

Operações com bits (resumo)

Operações mais importantes

- if (x & (1<<2)) ...
  - Testa se o bit 2 está setado
- x = x | (1<<2)
  - Liga o bit 2

Outras operações

- x = x & ~(1<<2)
  - Desliga o bit 2
- x = x << 5
  - Desloca 5 bits para a esquerda
- x = x >> 5
  - Desloca 5 bits para a direita

Motivação

Problema hipotético

- Considere uma coleção de livros
  - Livro 0 - O guia do mochileiro das galáxias (156 páginas)
  - Livro 1 - O restaurante no fim do universo (173 páginas)
  - Livro 2 - A vida, o universo e tudo mais (160 páginas)
  - Livro 3 - Até mais, e obrigado pelos peixes! (142 páginas)
  - Livro 4 - Praticamente inofensiva (191 páginas)
  - Livro 5 - E tem outra coisa... (366 páginas)
- Existe um subconjunto de livros totalizando 471 páginas?
  - Ideia: testar todos os subconjuntos possíveis!
  - Mas como iterar sobre os subconjuntos?

Ideia da solução

Números como conjuntos

- $x = 0$ (0000.0000) $\Longrightarrow$ nenhum livro
- $x = 1$ (0000.0001) $\Longrightarrow$ livro 0
- $x = 2$ (0000.0010) $\Longrightarrow$ livro 1
- $x = 3$ (0000.0011) $\Longrightarrow$ livros 1 e 0
- $x = 4$ (0000.0100) $\Longrightarrow$ livro 2
- $x = 5$ (0000.0101) $\Longrightarrow$ livros 2 e 0
- $x = 6$ (0000.0110) $\Longrightarrow$ livros 2 e 1
- $x = 7$ (0000.0111) $\Longrightarrow$ livros 2, 1 e 0
- $x = 8$ (0000.1000) $\Longrightarrow$ livro 3
- $x = 9$ (0000.1001) $\Longrightarrow$ livros 3 e 0
- ...
- $x = 62$ (0011.1110) $\Longrightarrow$ livros 5, 4, 3, 2 e 1
- $x = 63$ (0011.1111) $\Longrightarrow$ livros 5, 4, 3, 2, 1 e 0

Implementação da solução

```
vector<int> pages = {156, 173, 160, 142, 191, 366};
int n = pages.size();
for (int mask = 0; mask < (1<<n); ++mask) {
    int sum = 0;
    for (int i = 0; i < n; ++i)
        if (mask & (1<<i))
            sum += pages[i];
    if (sum == 471)
        cout<<"found: "<<mask<<endl;
}
```

Permutações

Permutações

Listando permutações:

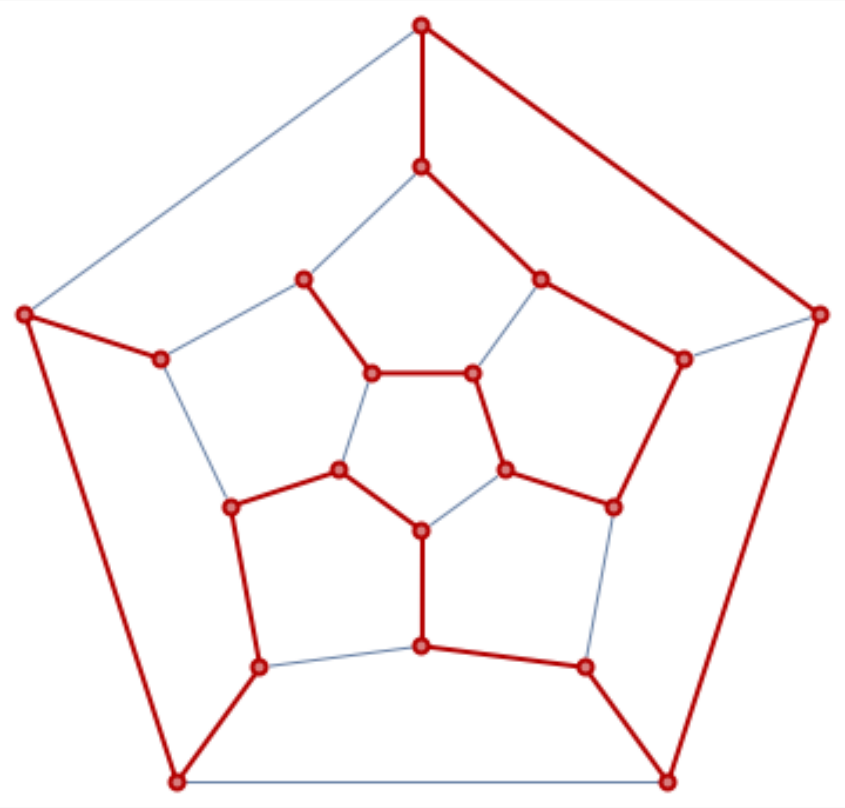
```
vector<int> vec = {1, 2, 3, 4, 5};
int n = vec.size();

// obs: o vetor precisa estar ordenado no começo
// se necessário, chame: sort(vec.begin(), vec.end());
do {
    for (int i = 0; i < n; ++i)
        cout << vec[i] << ' ';
    cout << endl;
} while (next_permutation(vec.begin(), vec.end()));
```

Exemplos práticos

Caminho hamiltoniano

Existe um caminho que passa por cada vértice exatamente uma vez?



Exemplos práticos

Listando primos

Liste todos os primos que não possuem dígitos repetidos.

```
set<int> find_primes(int len) {
    set<int> s;
    string digits = "0123456789";
    do {
        int x = to_integer(digits.substr(0, len));
        if (is_prime(x))
            s.insert(x);
    } while (next_permutation(digits.begin(), digits.end()));
    return s;
}
```

Maior primo

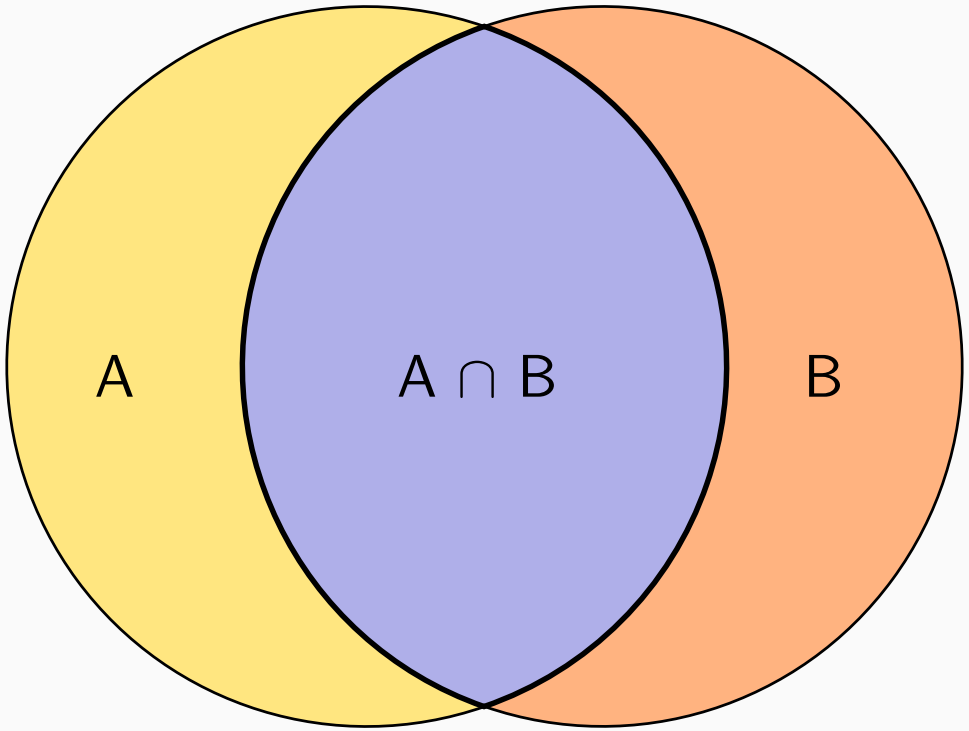
Qual é o maior número primo que possui todos os dígitos distintos?

Inclusão–Exclusão

Inclusão–Exclusão

Princípio para dois conjuntos

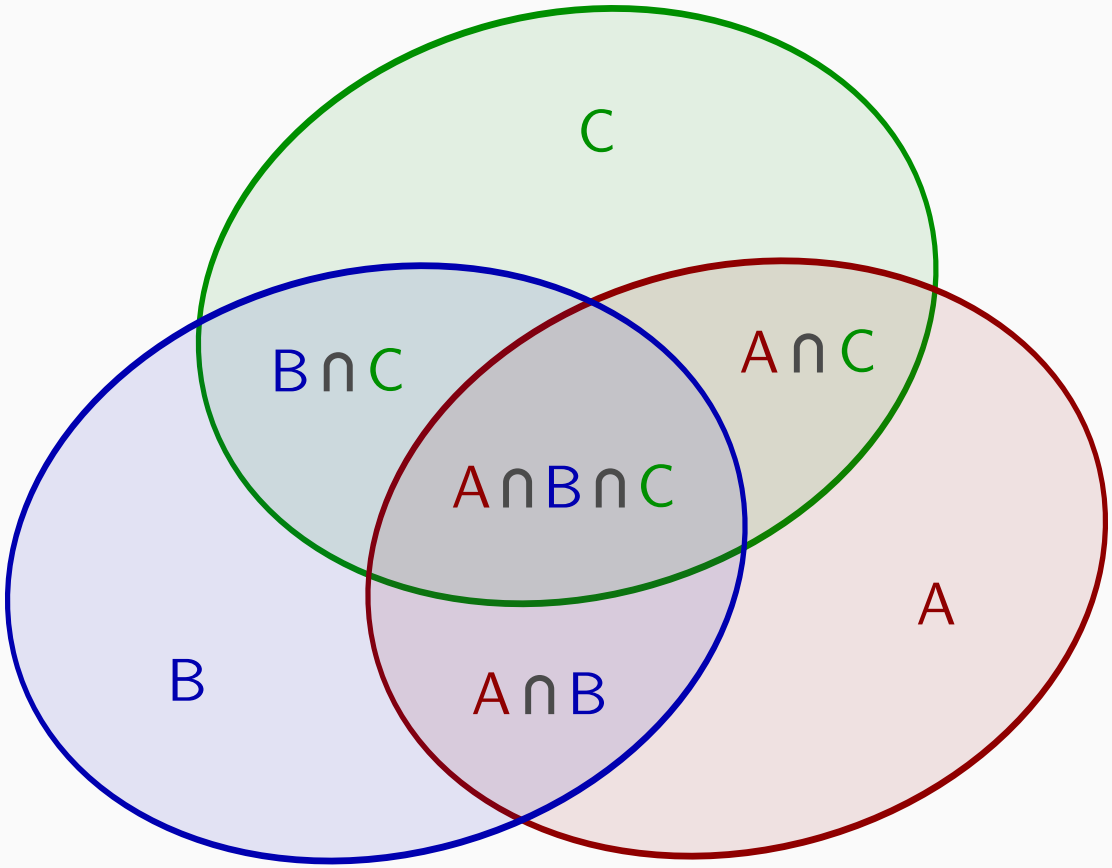
$$|A \cup B| = |A| + |B| - |A \cap B|$$



Inclusão–Exclusão

Princípio para três conjuntos

$$|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |A \cap C| - |B \cap C| + |A \cap B \cap C|$$



Inclusão–Exclusão

Princípio geral (versão 1)

$$\left|\bigcup_{i=1}^{n} A_i\right| = \sum_{k=1}^{n} (-1)^{k+1} \left(\sum_{1 \le i_1 < \cdots < i_k \le n} |A_{i_1} \cap \cdots \cap A_{i_k}| \right)$$

Código

```
int count = 0;
for (int mask = 1; mask < (1<<n); ++mask)
    if (__builtin_popcount(mask) % 2 == 1)
        count += f(mask);
    else
        count -= f(mask);
```

Inclusão–Exclusão

Princípio geral (versão 2)

$$\left|\bigcup_{i=1}^{n} A_i\right| = \sum_{\emptyset \neq J \subseteq \{1,2,\ldots,n\}} (-1)^{|J|-1} \left|\bigcap_{j \in J} A_j\right|$$

Código

```
int count = 0;
for (int mask = 1; mask < (1<<n); ++mask)
    if (__builtin_popcount(mask) % 2 == 1)
        count += f(mask);
    else
        count -= f(mask);
```

Exemplo prático (versão 1)

Quantos números de 1 a 10^9 são múltiplos de 3, 5, 7 ou 11?

Solução

```
vector<int> vec = {3, 5, 7, 11};
int n = vec.size();
int LIMIT = 1000000000; // 10^9
int count = 0;
for (int mask = 1; mask < (1<<n); ++mask) {
    int product = 1;
    for (int i = 0; i < n; ++i)
        if (mask&(1<<i))
            product *= vec[i];
    if (__builtin_popcount(mask) % 2 == 1)
        count += LIMIT / product;
    else
        count -= LIMIT / product;
}
```

Exemplo prático (versão 2)

Quantos números de 1 a 10^9 são múltiplos de 42, 54, 137 ou 201?

- Números não são mais coprimos

Solução

```
vector<int> vec = {42, 54, 137, 201};
int n = vec.size();
int LIMIT = 1000000000; // 10^9
int count = 0;
for (int mask = 1; mask < (1<<n); ++mask) {
    int lcm = 1;
    for (int i = 0; i < n; ++i)
        if (mask&(1<<i))
            lcm = lcm * vec[i] / __gcd(lcm, vec[i]);
    if (__builtin_popcount(mask) % 2 == 1)
        count += LIMIT / lcm;
    else
        count -= LIMIT / lcm;
}
```